

AWS GENERATIVE AI PROFESSIONAL

1. Foundation Models and Responsible Adoption

Detailed tutorial with metaphor, worked architecture case, design-first contract, failure laboratory, and certification practice

BUSINESS INVARIANT

No policy answer may be returned unless its claims are supported by current, authorized evidence for the customer context.

Certification alignment: Domain 1 - Foundation Models and AI Fundamentals

Tutorial promise: You will move from concept to an architecture decision, then prove the decision through contracts, tests, operational evidence, and scenario questions.

1. Learning Map and Mental Model

Foundation models are reusable probabilistic systems trained on broad data. The architect does not select a model because it sounds impressive in a demo. The decision starts with the business task, required quality, latency, data sensitivity, integration boundaries, cost ceiling, and the consequences of a wrong answer. Prompting, retrieval-augmented generation, fine-tuning, and deterministic code solve different problems. A responsible design assigns each concern to the mechanism that can prove it.

PLAIN ENGLISH

Think of a foundation model as a talented new colleague who has read an enormous library but has not memorized your latest company handbook. The colleague can explain and draft beautifully, yet still needs the approved handbook, a clear job description, and a supervisor for high-impact decisions.

MEMORY HOOK

The model is brilliant, fast, and occasionally as confident as someone giving directions after looking at the map upside down.

Learning outcomes

- Explain the topic to a nontechnical stakeholder without losing the control boundary.
- Identify the accountable actor, protected business truth, and prohibited AI action.
- Select AWS capabilities from requirements instead of starting with a service list.
- Design the normal workflow and the safe-failure workflow.
- Specify measurable evidence that can accept or reject the solution.

Core vocabulary

Term	Meaning in this tutorial
Business outcome	The measurable value the organization expects.
Invariant	No policy answer may be returned unless its claims are supported by current, authorized evidence for the customer context.
Probabilistic component	A model output that may vary even when the request appears similar.
Deterministic boundary	Code, policy, schema, or approval that produces a repeatable enforceable decision.
Evidence contract	Citation precision, grounded-answer rate, P95 latency, zero unauthorized disclosures, and traced escalation records.

2. Step-by-Step Architecture Tutorial

Begin with intent, then introduce technology only when a requirement needs a capability.

1	Frame the business problem A bank builds a policy assistant. Amazon Bedrock generates explanations, while a Knowledge Base retrieves only effective policy versions. Deterministic code checks customer jurisdiction, product eligibility, citation freshness, and confidence. Weak evidence produces NO_ANSWER or human escalation, never improvised financial advice.
2	Freeze the invariant No policy answer may be returned unless its claims are supported by current, authorized evidence for the customer context.
3	Define authority The accountable human or deterministic service owns the final business state. The AI assists only within this rule: Use RAG for changing policy knowledge; keep eligibility and authorization in deterministic code.
4	Design the interface contract Specify identity, required fields, schemas, idempotency, accepted and rejected responses, status transitions, and audit identifiers before implementation.
5	Separate fast intake from variable work Choose synchronous processing only for bounded validation and durable acceptance. Use asynchronous coordination when latency, bursts, or retries vary. Candidate services: Amazon Bedrock, Knowledge Bases for Amazon Bedrock, Amazon S3, AWS Lambda, Amazon CloudWatch, AWS KMS
6	Add deterministic gates Validate authorization, schema, policy, calculations, citations, safety, and allowed state transitions outside the model.
7	Define safe failure Fail closed on missing, stale, conflicting, or unauthorized evidence.
8	Accept with evidence Citation precision, grounded-answer rate, P95 latency, zero unauthorized disclosures, and traced escalation records.

3. Worked Case Study

CASE CONTEXT

A bank builds a policy assistant. Amazon Bedrock generates explanations, while a Knowledge Base retrieves only effective policy versions. Deterministic code checks customer jurisdiction, product eligibility, citation freshness, and confidence. Weak evidence produces NO_ANSWER or human escalation, never improvised financial advice.

Architecture reasoning

Question	Professional decision
What is business truth?	No policy answer may be returned unless its claims are supported by current, authorized evidence for the customer context.
Where does AI add value?	Use RAG for changing policy knowledge; keep eligibility and authorization in deterministic code.
What must AI never own?	Authorization, identity, official calculations, irreversible state changes, and final release authority unless the contract explicitly assigns and proves them.
How does the system fail?	Fail closed on missing, stale, conflicting, or unauthorized evidence.
What proves readiness?	Citation precision, grounded-answer rate, P95 latency, zero unauthorized disclosures, and traced escalation records.

End-to-end walkthrough

- The actor authenticates and submits a schema-valid request with a correlation identifier and idempotency key.
- The API enforces authorization and business preconditions before recording durable acceptance.
- The workflow obtains only authorized, versioned context and calls the selected model within latency, token, and safety budgets.
- Deterministic controls verify structure, grounding, calculations, permissions, and allowed state transition.
- The system records PASS, REVIEW, RETRY, or REJECT with evidence and exposes the status to the actor.
- An accountable owner reviews exceptions; monitoring detects threshold breaches and can stop or roll back the release.

4. Design-First Contract and Test Plan

Plain-English contract

WHEN	An authenticated, authorized actor submits a request that satisfies the published input contract.
THE SYSTEM PROMISES	Use RAG for changing policy knowledge; keep eligibility and authorization in deterministic code.
THE SYSTEM REFUSES	No policy answer may be returned unless its claims are supported by current, authorized evidence for the customer context.
IF SOMETHING FAILS	Fail closed on missing, stale, conflicting, or unauthorized evidence.

Acceptance tests

Test	Injection or stimulus	Pass condition
Contract validation	Remove a required field or change its type.	Rejected before model invocation or side effect; reason is traceable.
Authorization	Use another role or tenant identity.	Access denied; no protected context is retrieved or disclosed.
Idempotency	Replay the same request or event.	One logical business result; replay returns the known state.
AI quality	Return schema-invalid, unsupported, or unsafe output.	Deterministic gate blocks release and routes RETRY, REVIEW, or REJECT.
Dependency failure	Throttle or time out an AWS or model dependency.	Fail closed on missing, stale, conflicting, or unauthorized evidence.
Evidence	Citation precision, grounded-answer rate, P95 latency, zero unauthorized disclosures, and traced escalation records.	Every threshold is observable, attributable, and linked to a release.

5. Operations, Evidence, and Well-Architected Review

Pillar	Questions to ask
Operational excellence	Can operators trace a request, reproduce the decision, run the failure playbook, and roll back?
Security	Is identity enforced before retrieval and tool use? Are secrets, data, and logs protected?
Reliability	Are retries bounded and idempotent? Does every accepted item reach a terminal state?
Performance efficiency	Which latency component dominates? Is the selected model and retrieval pattern appropriate?
Cost optimization	Is cost measured per accepted outcome? Are tokens, caching, batching, and capacity governed?
Sustainability	Is the smallest capable model used? Does the design scale to zero or process efficiently when idle?

Evidence dashboard

Layer	Measure	Why it matters
Business	Outcome KPI and protected invariant	Proves the system creates value without violating business truth.
Application	Acceptance latency, error rate, terminal-state completion	Separates user experience from downstream AI processing.
AI quality	Grounding, factuality, safety, variance	Measures probabilistic quality directly.
Security	Denied access, leakage tests, suspicious prompts	Shows the authority boundary remains intact.
Operations	Queue age, retries, DLQ, alarm response, rollback time	Shows safe recovery rather than silent failure.
Economics	Cost per accepted outcome and model utilization	Prevents cheap tokens from masking expensive failed work.

6. Certification-Style Knowledge Check

Choose the best architectural response. Answers are on the next page.

1. Which design choice BEST protects the business requirement in this tutorial?

- A. Allow the model to make the final decision when it sounds confident.
- B. Use RAG for changing policy knowledge; keep eligibility and authorization in deterministic code.
- C. Optimize speed first and add authority controls after launch.
- D. Share cached answers across users without authorization.

2. The AI dependency times out after the business request is accepted. What should the system do?

- A. Delete the request and ask the user to retry from memory.
- B. Mark the workflow successful without processing.
- C. Repeat every side effect without an idempotency key.
- D. Fail closed on missing, stale, conflicting, or unauthorized evidence.

3. Which evidence package MOST strongly demonstrates production readiness?

- A. One successful demonstration.
- B. Citation precision, grounded-answer rate, P95 latency, zero unauthorized disclosures, and traced escalation records.
- C. A fluent response screenshot.
- D. A claim that the largest model should be reliable.

4. A generated response contains an unsupported but plausible claim. What is the BEST action?

- A. Publish it because users probably will not notice.
- B. Raise the temperature and regenerate creatively.
- C. Block the business side effect and route the result through the defined review or failure path.
- D. Remove the citation requirement.

5. What is the correct interpretation of these services: Amazon Bedrock, Knowledge Bases for Amazon Bedrock, Amazon S3, AWS Lambda, Amazon CloudWatch, AWS KMS?

- A. Managed services remove the need for business ownership.
- B. The model should dynamically grant its own IAM permissions.
- C. They implement capabilities, while contracts, deterministic controls, tests, and accountable owners establish trust.
- D. Their use guarantees zero hallucinations.

7. Answers, Explanations, and Teaching Prompts

1. Correct answer: B

The architecture should use rag for changing policy knowledge; keep eligibility and authorization in deterministic code. This directly protects the invariant: No policy answer may be returned unless its claims are supported by current, authorized evidence for the customer context.

Key knowledge: Business authority and invariant

Teaching prompt: Ask which business fact, authority, or evidence would be lost if the selected control were removed.

2. Correct answer: D

The documented safe-failure response is: Fail closed on missing, stale, conflicting, or unauthorized evidence. Accepted state must not disappear because a probabilistic dependency is unavailable.

Key knowledge: Safe failure and durable state

Teaching prompt: Ask which business fact, authority, or evidence would be lost if the selected control were removed.

3. Correct answer: B

The required evidence is measurable and repeatable: Citation precision, grounded-answer rate, P95 latency, zero unauthorized disclosures, and traced escalation records.

Key knowledge: Measurable evidence

Teaching prompt: Ask which business fact, authority, or evidence would be lost if the selected control were removed.

4. Correct answer: C

Unsupported output is a controlled failure. Grounding and deterministic validation must prevent it from becoming business truth.

Key knowledge: Grounding and deterministic validation

Teaching prompt: Ask which business fact, authority, or evidence would be lost if the selected control were removed.

5. Correct answer: C

AWS services provide capabilities, scaling, and operational mechanisms. The business contract still defines authority, safe failure, and measurable acceptance.

Key knowledge: Managed services within a governed architecture

Teaching prompt: Ask which business fact, authority, or evidence would be lost if the selected control were removed.

Final recap

- Business invariant: No policy answer may be returned unless its claims are supported by current, authorized evidence for the customer context.
- Architecture decision: Use RAG for changing policy knowledge; keep eligibility and authorization in deterministic code.
- Safe failure: Fail closed on missing, stale, conflicting, or unauthorized evidence.
- Acceptance evidence: Citation precision, grounded-answer rate, P95 latency, zero unauthorized disclosures, and traced escalation records.

EXAM TIP

Prefer the option that satisfies the explicit requirement, preserves authority, fails safely, minimizes unnecessary operational burden, and can be proven with measurable evidence.

Original teaching material. Verify current AWS service documentation and the current certification exam guide before production or examination decisions.